Intelligent Project Management System

For Undergraduate Students

IPMS
R26-IT-095
SLIIT 2026

Project ID: R26-IT-095

Department of Information Technology · SLIIT · 2026

Our Team



Bandara N.D.M.
IT22114662

Team Formation & Feasibility



Lekamge R.L.M.D.
IT22117014

Project Planning & Scheduling



Kulathunga S.A.I.K.
IT22109576

Risk Monitoring & Analytics



Rashmika R.M.D.
IT22101624

Quality Assessment

CONTENT

01 Introduction & Problem Statement

02 Research Gap

03 System Overview & Objectives

04 **Component 01** — Team Formation (Bandara N.D.M.)

05 **Component 02** — Project Planning (Lekamge R.L.M.D.)

06 **Component 03** — Risk Monitoring (Kulathunga S.A.I.K.)

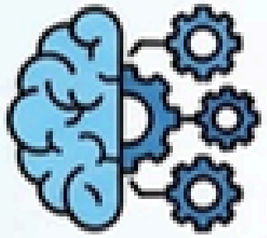
07 **Component 04** — Quality Assessment (Rashmika R.M.D.)

08 Technologies & Validation Metrics

09 Gantt Chart · Commercialization · Budget

01. Introduction

What is IPMS?

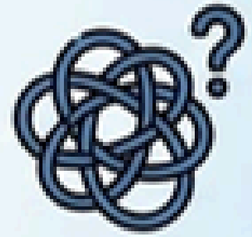


An AI-powered system managing the full lifecycle of undergraduate group projects.

Addresses the full lifecycle of group projects from team formation to final report assessment.

Built as 4 integrated components working together on a single platform.

What is the Problem?

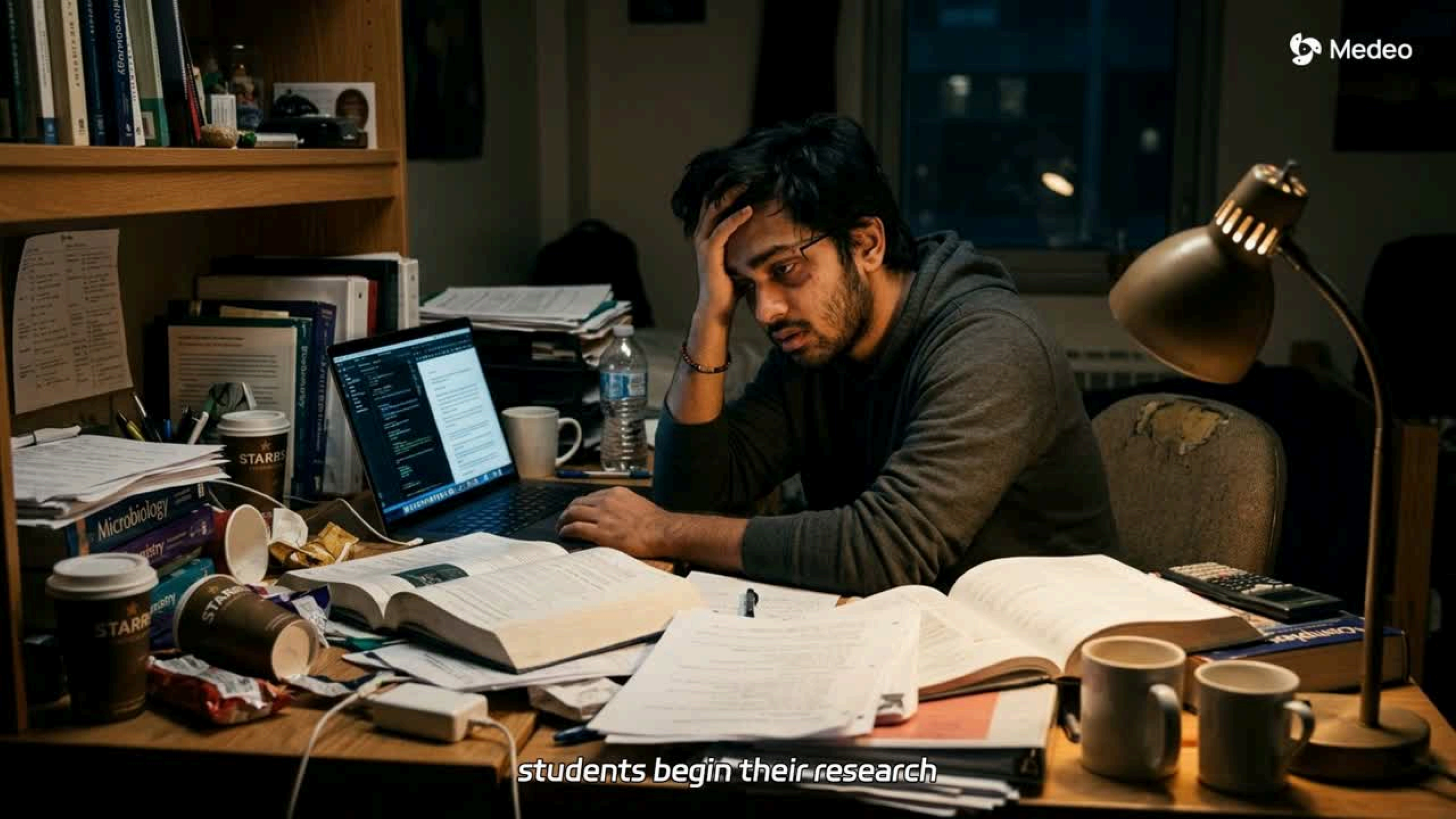


Undergraduate group projects fail due to poor team formation, weak planning, undetected risks, and inconsistent feedback.

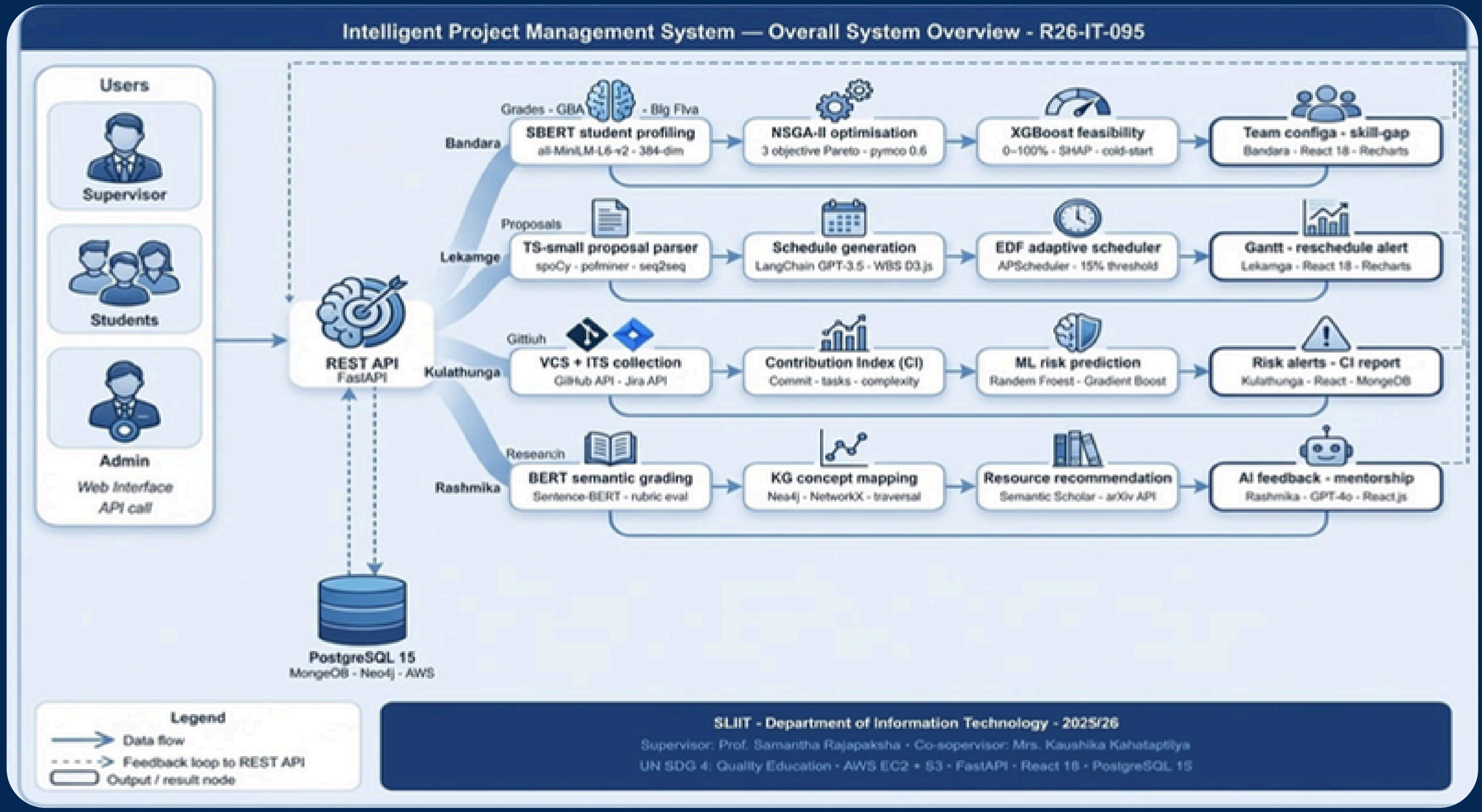
All current processes are manual, informal, and lack data-driven intelligence.

No existing tool covers the full project lifecycle end-to-end.

The Problem & Proposed Solution


Medeo
Microbiology
STARBUCKS
students begin their research

System Overview



Existing Solutions and Their Limitations

Aspect	Existing Tools	Gap Identified	Proposed IPMS
Team Formation	Random / CATME self-report	No NLP profiling, no multi-objective optimisation	**SBERT + NSGA-II Pareto-optimal 3-objective**
Topic Feasibility	None	Mismatches found after work begins	**XGBoost P(success) 0–100% + SHAP**
Project Planning	Manual Gantt tools only	No NLP parsing, no adaptive scheduling	**T5 seq2seq + EDF velocity rescheduler**
Contribution Tracking	Raw GitHub commit counts	No quality/complexity weighting	**Weighted Contribution Index (CI)**
Quality Feedback	Turnitin / Moodle only	No concept diagnosis, no resources	**BERT + Neo4j KG + GPT-4o feedback**



BANDARA N.D.M.

IT22114662

Intelligent Team Formation
&
Topic Feasibility Analysis

Problem and Solutions

Problem

Teams are formed randomly with no consideration of skills, personality diversity, or whether the team can actually handle their chosen research topic.

Proposed Solution

SBERT encodes each student's skills into a 384-dim vector, NSGA-II builds the most balanced team, and XGBoost predicts topic readiness with a 0–100% success score before the project starts.

Research Gap

Aspect	Random/CATME	Proposed IPMS
Team Formation	No data / self-report scales	**SBERT semantic skill vectors**
Skill Profiling	None / Dunning-Kruger bias	**Cosine similarity ≥ 0.80**
Optimisation	None / single-objective	**NSGA-II 3-objective Pareto-optimal**
Personality	Not considered	**Big Five diversity maximisation**
Topic Feasibility	None	**XGBoost P(success) 0–100%**
Explainability	None	**SHAP feature-level explanation**

Objectives & Sub-objectsives



Main Objective

Design and implement an AI-powered system that automatically forms balanced student groups and evaluates project topic feasibility using semantic skill profiling and multi-objective optimization. The system ensures optimal team composition and validates team—topic suitability using machine learning models before the project begins.

Sub Objectives



SO1: SBERT Student Vector Profiling Engine

To Build the SBERT Student Vector Profiling Engine



SO2: NSGA-II Multi-Objective Team Optimization Engine

To Implement NSGA-II Multi-Objective Team Optimization Engine



SO3: XGBoost Topic Feasibility Predictor & Cold-Start Validator

To Develop XGBoost Topic Feasibility Predictor and Cold-Start Validator



SO4: IPMS Integration API & Supervisor Dashboard

To Develop IPMS Integration API and Supervisor Dashboard



SO5: System Performance Evaluation & Validation

To Evaluate System Performance and Validate Outcomes

Methodology & Validation

Methodology

- Design Science Research (DSR) approach
- SBERT encodes student profile → 384-dim skill vector
- NSGA-II forms Pareto-optimal teams (Pop=200, Gen=500, 3 objectives)
- XGBoost predicts P(success) per team-topic pair
- Cold-Start Validator flags skill gaps element-wise

Data Validation

- Cosine similarity ≥ 0.80 (SBERT profiling)
- Expert rating ≥ 4.0/5.0 (team quality)
- F1 ≥ 0.82, AUC-ROC ≥ 0.85 (XGBoost)
- Formation speed ≤ 60s for 100 students

Availability & Ethics

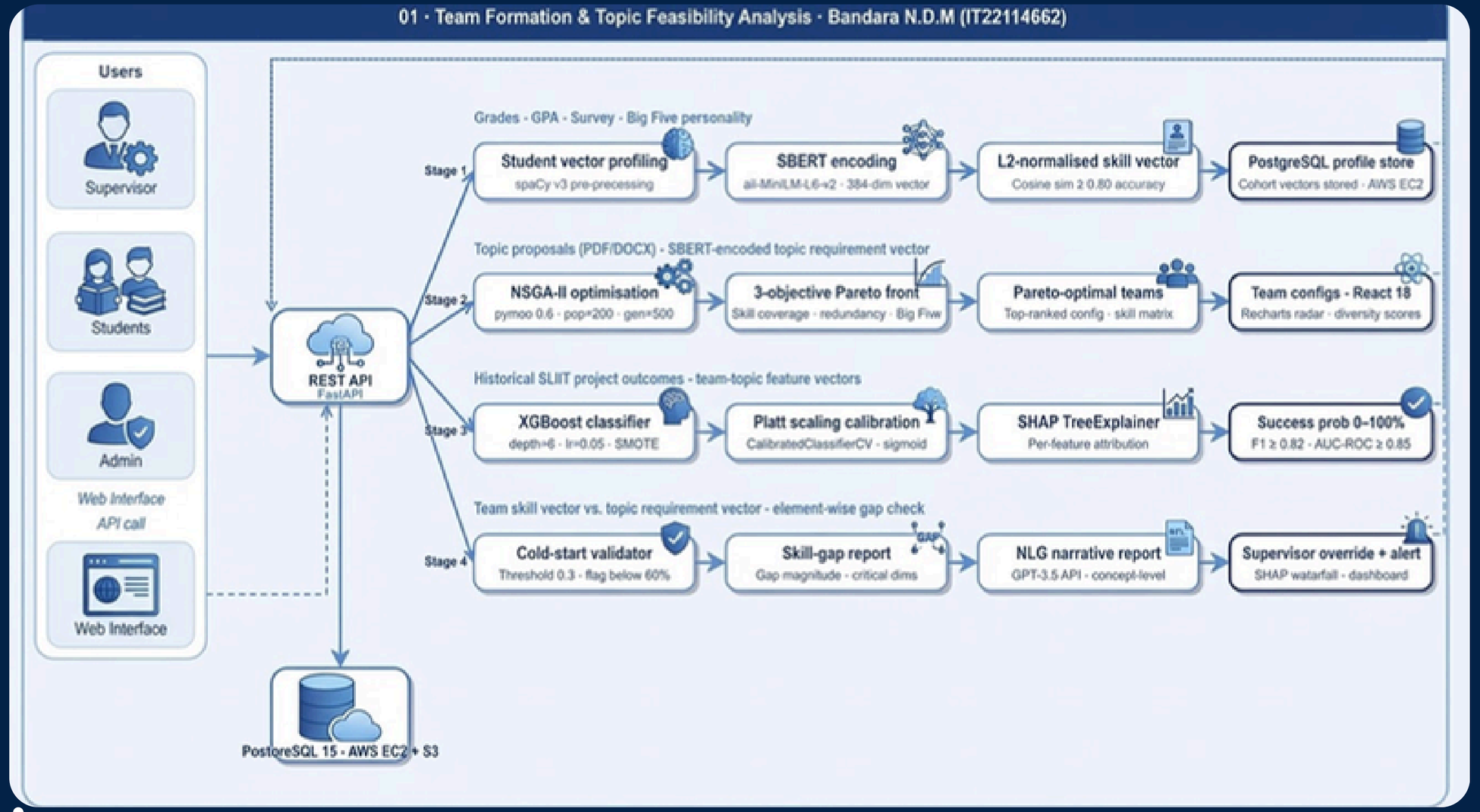
Data Availability

- xAPI-Edu-Data — Kaggle 2016 (480 records)
- Higher Education Performance — Kaggle 2022
- Student Performance Factors — Kaggle 2024 (6,607 records)

Ethics

- SLIIT IT Department ethics board approval
- Full data anonymisation before processing
- Written informed consent from all participants

System Overview





Lekamge R.L.M.D

IT22117014

Generative Project Planning & Adaptive Scheduling

Problem and solutions

Problem

Students manually build Gantt charts in week one, abandon them at the first delay, and never update them — leaving projects with no real plan.

Proposed Solution

T5 reads the proposal and auto-generates a full WBS and Gantt chart, then an EDF scheduler continuously monitors progress and automatically reschedules when delays are detected.

Research Gap

Aspect	Existing Tools	Proposed IPMS
Planning Input	Fully manual entry	**Auto-generated from proposal (NLP)**
Proposal Understanding	None	**T5-based phase & deliverable extraction**
WBS Generation	Manual	**Automated, dependency-inferred**
Schedule Adaptation	None (static)	**Real-time EDF rescheduler**
Delay Response	Manual update required	**Automated velocity-based correction**
Integration	Standalone tools	**REST API → IPMS modules**

Objectives & Sub-objectsives

Main Objective:
Design and implement an AI-powered system that automatically generates project plans and schedules by using NLP to analyze project proposals and extract phases and deliverables. The system creates Work Breakdown Structures and Gantt charts and continuously adjusts using real-time team performance feedback.

01 SO1: Implement NLP-based Proposal Parsing Engine

02 SO2: Build the Schedule Generation Engine

03 SO3: Develop the Adaptive Feedback Loop Scheduler

04 SO4: Develop IPMS Integration API

05 SO5: Evaluate System Performance and Validate Outcomes

Methodology & Validation

Methodology

- Design Science Research (DSR) approach
- spaCy v3 parses proposal PDF/DOCX by section
- T5-small seq2seq extracts phases, deliverables, time estimates
- LangChain + GPT-3.5 generates WBS with dependency ontology
- APScheduler polls every 24h → EDF rescheduler triggered if delay > 15%

Data Validation

- ROUGE-L ≥ 0.80, F1 ≥ 0.85 (T5 parser)
- MAE < 1.8 days (XGBoost duration estimator)
- ≥ 25% deadline adherence improvement vs static planning
- Expert schedule quality rating ≥ 4.0/5.0

Availability & Ethics

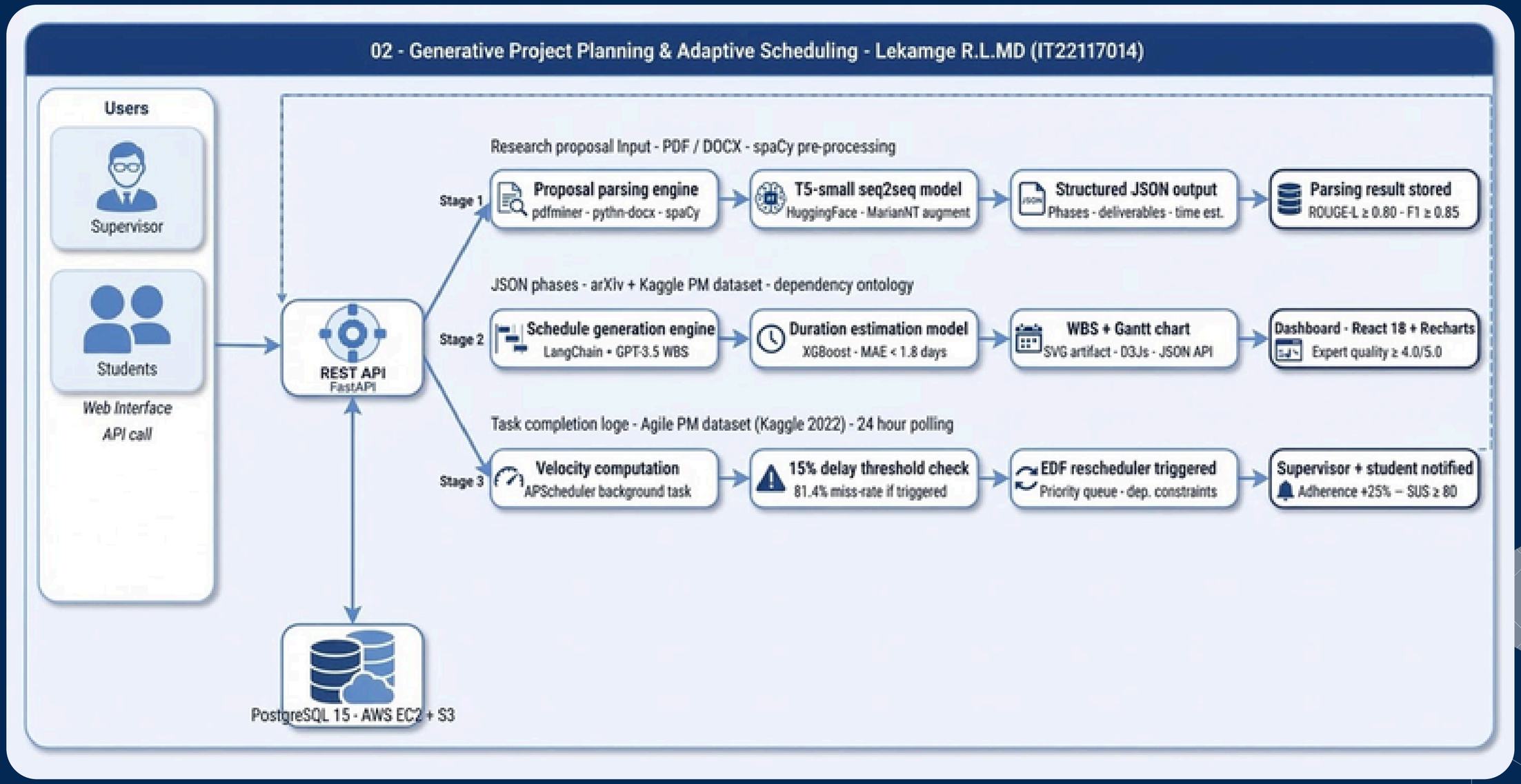
Data Availability

- arXiv Research Papers — Kaggle 2020 (~12,000 CS papers)
- Project Management Dataset — Kaggle 2024
- Agile Software Development Dataset — Kaggle 2022 (3,259 sprints)

Ethics

- arXiv Research Papers — Kaggle 2020 (~12,000 CS papers)
- Project Management Dataset — Kaggle 2024
- Agile Software Development Dataset — Kaggle 2022 (3,259 sprints)

System Overview





Kulathunga S.A.I.K

IT22109576

Predictive Risk Monitoring and Contribution Analytics

Problem and Solutions

Problem

Main problem :How can we detect early signs of project risks or failure?

- How can students be alerted when a project is at risk?
- How can supervisors track each student's contribution?
- How watch progress during the project?

Proposed Solution

- Intelligent monitoring using project activity data
- Calculate Contribution Index (CI)
- Predict risks using machine learning
- Monitor progress with real-time dashboards

IT22109576 | Kulathunga S.A.I.K

Research Gap

Aspect	Existing Tools	Research Gap	Proposed IPMS
Contribution Assessment	Tools measure commits and task counts.	Do not consider quality or complexity of work.	**Calculates a Contribution Index (CI) for fair evaluation.**
Predictive Monitoring	ML models mainly used for industry projects.	Not suitable for academic student projects.	**Uses ML models designed for student projects.**
Risk Detection	Supervisors depend on manual monitoring.	Problems are identified too late.	**Predicts risks early and sends alerts.**
Data Integration	Project tools analyze data separately.	Lack of combined analysis from multiple sources.	**Integrates VCS and ITS data for better monitoring.**

Objectives & sub-objectsives

Main Objective:

Design and implement an **Intelligent Project Management System (IPMS)** that integrates data from **Version Control Systems and Issue Tracking Systems** to monitor project risks in real time and fairly evaluate individual student contributions in undergraduate software development projects.

Sub Objectives

- **SO1:** develop a **Contribution Index (CI)**
- **SO2:** implement **machine learning models**
- **SO3:** design and integrate **real-time analytics dashboards**
- **SO3:** design and integrate **real-time analytics dashboards**
- **SO4:** validate the **IPMS performance**
- **SO5:** ensure alignment with **academic assessment standards**

Data Availability & Technology

Data Availability

- GitHub Repositories Dataset — Kaggle 2023 (20,000+ repos)
- Software Defect Prediction (PROMISE) — Kaggle 2023
- Jira Software Projects Dataset — Kaggle 2023 (50,000+ issues)

Technologies

- Python for data processing
- Scikit-learn for machine learning models
- GitHub API & Jira API for project activity data
- Web dashboards for visualization

Methodology, Validation & Ethics

Methodology

- Data collection from repositories
- Data processing and feature extraction
- Contribution analysis
- Risk prediction using ML

Ethics

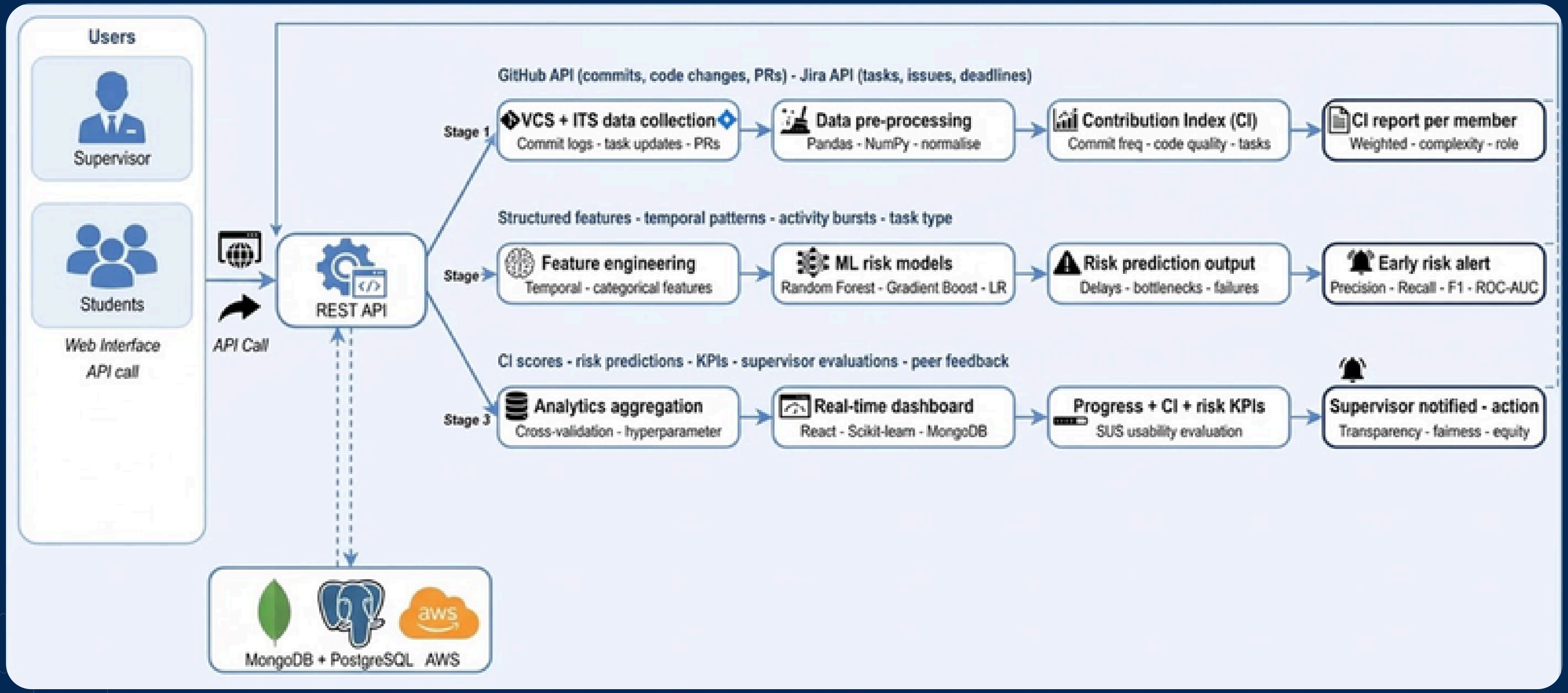
- Data anonymization
- Secure storage
- Participant consent

Validation

Precision, Recall, F1-Score

Compare CI with supervisor evaluation

System Overview





RASHMIKA R.M.D.

IT22101624

Automated Quality Assessment
&
Adaptive Mentorship

Probelm and solutions

Problem

Supervisors manually grade dozens of reports under time pressure and students receive vague scores with no guidance on what to read, learn, or improve.

Proposed Solution

BERT semantically grades each report section against a rubric, a Neo4j Knowledge Graph maps the exact weaknesses, and GPT-4o generates a personalised improvement report with recommended resources.

Research Gap

Aspect	Random/CATME	Proposed IPMS
Team Formation	No data / self-report scales	**SBERT semantic skill vectors**
Skill Profiling	None / Dunning-Kruger bias	**Cosine similarity ≥ 0.80**
Optimisation	None / single-objective	**NSGA-II 3-objective Pareto-optimal**
Personality	Not considered	**Big Five diversity maximisation**
Topic Feasibility	None	**XGBoost P(success) 0–100%**
Explainability	None	**SHAP feature-level explanation**

Research Objectives & sub-objectsives

Main Objective:
The goal of the project is to develop an **AI-based tool** that automatically evaluates the quality of undergraduate research reports, identifies areas where students are underperforming, and provides **personalized feedback** and **learning resources** to help them improve.

Module 1	Module 2	Module 3	Module 4	Module 5
SO1: Implement NLP-based Semantic Grading Engine	SO2: Construct Academic Knowledge Graph for Concept Mapping	SO3: Developing an AI-Powered Resource Recommendation Engine	SO4: Automate Structured Feedback Report Generation	SO5: Assess System Performance and Confirm Learning Outcome

Methodology & Validation

Methodology

- Design Science Research (DSR) approach
- spaCy v3 segments report into 6 rubric sections
- SBERT dual-encoder computes cos(V_student, V_rubric) → BERT head classifies Excellent/Good/Adequate/Poor
- Neo4j KG + NetworkX traversal identifies root-cause concept gaps

Data Validation

- Cohen's Kappa ≥ 0.80 vs 3 expert supervisor raters
- Concept mapping precision ≥ 85% (KG node mapping)
- Resource relevance ≥ 80% (supervisor rating)
- SUS ≥ 85 | Grading ≤ 3 min per report

Availability & Ethics

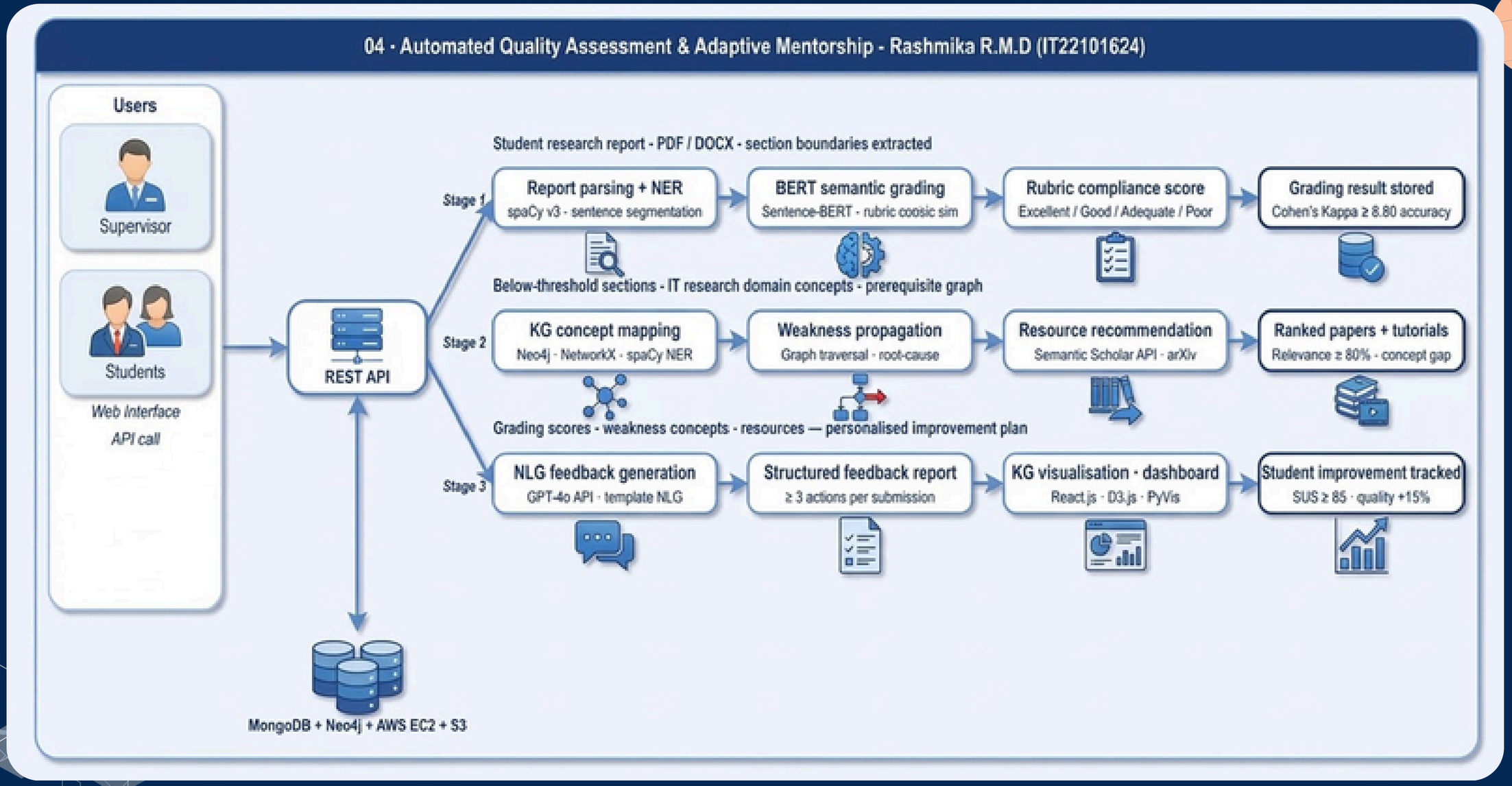
Data Availability

- ASAP Essay Scoring Corpus — Kaggle 2012 (12,976 essays)
- Students Performance in Exams — Kaggle 2018
- arXiv IT/CS Papers — Kaggle (5,000–10,000 papers)

Ethics

- SLIIT IT Department ethics clearance before any data collection
- Full anonymisation — no PII retained after processing
- Written informed consent from all student participants

System Overview



Commercialization Plan

Target Market & Customer

Primary Market:

- Higher education institutions in Sri Lanka and South/Southeast Asia running undergraduate STEM & IT programs.
- Immediate target: SLIIT → expansion to University of Moratuwa, NSBM, IIT Sri Lanka.
- Regional expansion: India, Malaysia, Singapore.

Secondary Market:

- EdTech platforms offering research supervision or project management services.
- Corporate graduate training programs requiring structured team and report evaluation.

Customer Persona:

- University departments, faculty supervisors, and project coordinators overseeing 50–500+ student projects simultaneously.
- Undergraduate students who benefit from AI-powered planning, fair contribution tracking, and personalized feedback.

Investment

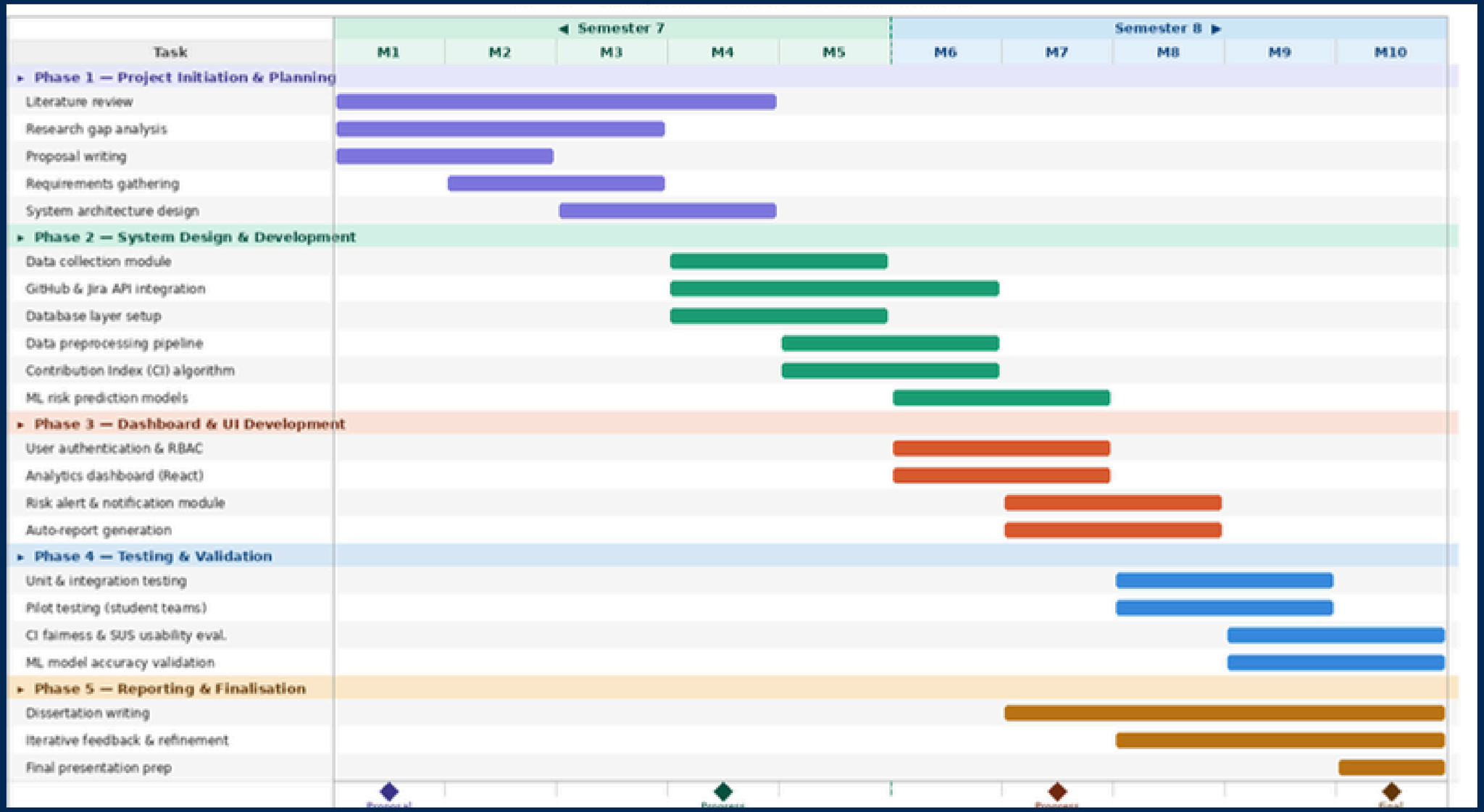
Item	Cost (LKR)
One-Time Setup Costs (Total: ~LKR 85,000)	
GPU Cloud — Model Training (all components)	60,000
Miscellaneous (Testing, annotation, tools)	20,000
Domain & SSL Certificate	5,000
Monthly Operational Costs (Total: ~LKR 135,000/month)	
AWS EC2 Instances (4 components)	~57,000
AWS S3 Storage	~8,000
PostgreSQL / Neo4j Managed Hosting	~24,000
OpenAI / HuggingFace APIs	~28,000
Semantic Scholar API (Premium)	5,000
Miscellaneous	~13,000

Pricing & How to Recover Investment

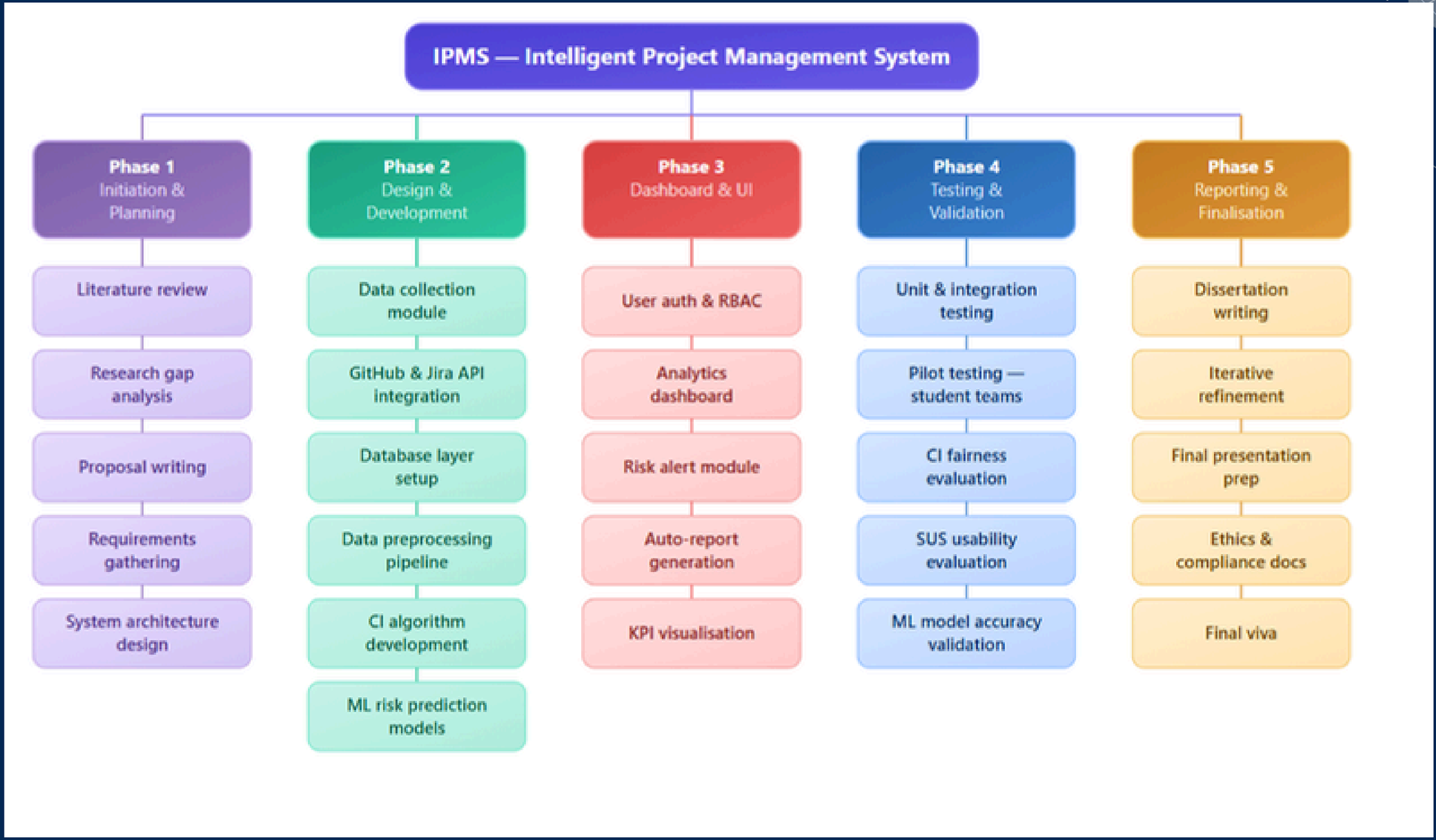
Revenue Model — SaaS Tiered Institutional Licensing

Plan	Features	Price
Basic Academic	Project monitoring, dashboards, contribution analytics	LKR 100,000-150,000/year
Premium Academic	Risk prediction, advanced analytics, automated reports	LKR 250,000-350,000/year
Enterprise University	Full IPMS, unlimited users, LMS integration, support	LKR 500,000-2,000,000/year

Gantt Chart


◀ Semester 7
Semester 8 ▶
Task
M1
M2
M3
M4
M5
M6
M7
M8
M9
M10
▸ Phase 1 — Project Initiation & Planning
Literature review
Research gap analysis
Proposal writing
Requirements gathering
System architecture design
▸ Phase 2 — System Design & Development
Data collection module
GitHub & Jira API integration
Database layer setup
Data preprocessing pipeline
Contribution Index (CI) algorithm
ML risk prediction models
▸ Phase 3 — Dashboard & UI Development
User authentication & RBAC
Analytics dashboard (React)
Risk alert & notification module
Auto-report generation
▸ Phase 4 — Testing & Validation
Unit & integration testing
Pilot testing (student teams)
CI fairness & SUS usability eval.
ML model accuracy validation
▸ Phase 5 — Reporting & Finalisation
Dissertation writing
Iterative feedback & refinement
Final presentation prep

Work breakdown structure



Q & A

Thank You!